

Fiche TP — Semaine 1 — Mise en place du projet

Développement web, classe passerelle

Durée

indicative : 4h

Configurer un environnement de développement fonctionnel et initialiser le dépôt Git du projet fil rouge **MiniForum**.

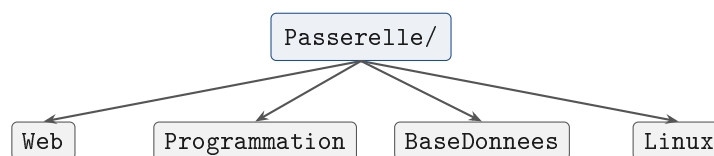
Ce TP suppose que les notions du polycopié *S01_polycop* (client/serveur, HTML/CSS/JS, Git) ont été vues en cours. Garde-le à portée de main.
Les instructions ci-dessous concernent **Windows**.

1 S'organiser : arborescence et sauvegarde

Avant d'installer quoi que ce soit, on met en place une organisation de fichiers claire et une sauvegarde automatique. Tu ne perdras jamais ton travail, et tu retrouveras toujours facilement tes cours et projets.

Créer l'arborescence des cours

1. Sur ton disque, ouvrir le répertoire Documents.
2. Y créer un répertoire Passerelle.
3. À l'intérieur de Passerelle, créer les sous-répertoires suivants :



Synchroniser automatiquement sur Google Drive

Afin de ne pas perdre ton travail en cas de perte, casse ou écrasement de disque, tu vas synchroniser automatiquement le répertoire Passerelle sur ton espace Google Drive personnel.

Étape 1 — Mettre en place la synchronisation

1. Télécharger et installer **Drive pour ordinateur** : google.com/intl/fr/drive/download
2. Se connecter avec ton **compte eduge**.
3. Appuyer deux fois sur *Suivant*, jusqu'à l'étape de sélection du dossier à synchroniser.
4. Cliquer sur **Ajouter un dossier**.



Capture d'écran — ajouter un dossier à synchroniser

5. Sélectionner le dossier Passerelle créé à l'étape précédente.
6. Vérifier que **seul** le dossier Passerelle est coché, puis cliquer sur *Suivant*.
7. Continuer en appuyant sur *Suivant*.



Capture d'écran — dossier Passerelle sélectionné

Attention

Ne synchronise que le dossier Passerelle. Ne coche pas *Desktop*, *Documents* ou *Downloads* : tu risquerais de synchroniser bien plus que nécessaire et de saturer ton espace Drive.

Étape 2 — Vérifier le bon fonctionnement

1. Dans ton navigateur, aller sur ton **compte eduge** puis ouvrir **Drive**.
2. Dans le Drive, cliquer sur *Ordinateurs* → *Mon ordinateur* → Passerelle.
3. Tu dois retrouver l'arborescence créée sur ton ordinateur.



Capture d'écran — vérification de l'arborescence côté Drive

4. Créer un fichier `test.txt` sur ton ordinateur, dans le dossier `Web`.
5. Vérifier qu'il apparaît bien côté Drive (après quelques secondes).

Ton travail est désormais sauvegardé automatiquement. Tu es prêt à attaquer l'année de manière organisée.

2 Installation des outils (Windows)

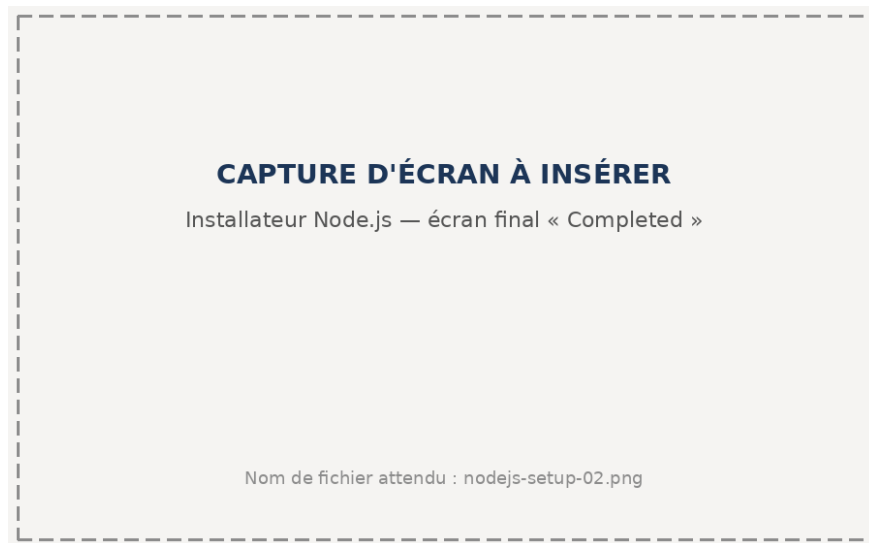
Node.js

1. Aller sur `nodejs.org` et télécharger la version **LTS** pour Windows (fichier `.msi`).
2. Lancer l'installateur téléchargé, accepter la licence, cliquer sur *Next*.
3. Dossier de destination : garder celui proposé par défaut.
4. Écran *Custom Setup* : laisser toutes les cases cochées, en particulier **npm package manager** et **Add to PATH**.



Capture d'écran — écran « Custom Setup » de l'installateur Node.js

- Écran *Tools for Native Modules* : laisser décoché (non nécessaire pour ce cours), cliquer sur *Next*.
- Cliquer sur *Install*, patienter, puis *Finish*.



Capture d'écran — écran final de l'installateur Node.js

Vérification

Ouvrir une invite de commande (*cmd* ou *PowerShell*) et exécuter `node -v` puis `npm -v` : chaque commande doit afficher un numéro de version.

Git

- Aller sur git-scm.com/download/win et télécharger l'installateur 64-bit.
- Lancer l'installateur, accepter la licence, garder le dossier de destination par défaut.
- Écran de sélection des composants : garder les cases cochées par défaut.
- Écran *Choosing the default editor used by Git* : sélectionner **Visual Studio Code** dans la liste déroulante.



Capture d'écran — choix de l'éditeur par défaut

5. Écran *Adjusting the name of the initial branch* : choisir **main** comme nom de branche par défaut.
6. Écran *Adjusting your PATH environment* : garder l'option recommandée du milieu (*Git from the command line and also from 3rd-party software*).



Capture d'écran — réglage du PATH

7. Écran *Choosing the SSH executable* : garder par défaut (*Use bundled OpenSSH*).
8. Écran *Choosing HTTPS transport backend* : garder par défaut (*Use the OpenSSL library*).
9. Écran *Configuring the line ending conversions* : garder l'option par défaut (*Checkout Windows-style, commit Unix-style line endings*).



Capture d'écran — conversion des fins de ligne

10. Écrans suivants (*terminal emulator, git pull, credential helper*) : garder les options par défaut proposées.
11. Cliquer sur *Install*, puis *Finish*.

Vérification

Ouvrir une invite de commande et exécuter `git -version` : la commande doit afficher un numéro de version.

Visual Studio Code

1. Aller sur `code.visualstudio.com` et télécharger la version Windows.
2. Lancer l'installateur, accepter la licence, garder le dossier de destination par défaut.
3. Écran *Select Additional Tasks* : cocher **Add to PATH**, **Register Code as an editor for supported file types**, et les options *Open with Code* du menu contextuel.



Capture d'écran — écran « Select Additional Tasks »

4. Cliquer sur *Install*, puis *Finish* en laissant *Launch Visual Studio Code* coché.
5. Installer les extensions listées dans le polycopié (Prettier, ESLint, Live Server, REST Client, GitLens).



Capture d'écran — lancement de VS Code après installation

3 Mise en place du projet

1. **Vérifier les installations.** Ouvrir un terminal et exécuter :

```
1 node -v
2 git --version
```

Chaque commande doit afficher un numéro de version. Sinon, revoir la section 1 avant de continuer.

2. Créer le dossier du projet.

```
1 mkdir miniforum
2 cd miniforum
```

3. Créer les trois fichiers de base (index.html, style.css, app.js) directement depuis VS Code, ou avec touch sous macOS/Linux.

4. Reproduire l'exemple minimal vu en cours :

Listing 1: index.html

```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="UTF-8">
5   <title>MiniForum</title>
6   <link rel="stylesheet" href="style.css">
7 </head>
8 <body>
9   <h1>Bienvenue sur MiniForum</h1>
10  <button id="btn">Cliquez-moi</button>
11  <script src="app.js"></script>
12 </body>
13 </html>
```

Listing 2: style.css

```
1 button {
2   background-color: #1D3557;
3   color: white;
4   padding: 10px 20px;
5   border: none;
6   border-radius: 4px;
7   cursor: pointer;
8 }
```

Listing 3: app.js

```
1 const btn = document.getElementById('btn');
2 btn.addEventListener('click', () => {
3   alert('Bonjour depuis JavaScript !');
4 });
```

Vérifier dans le navigateur que le bouton affiche bien une alerte au clic.

4 Git et GitHub

Configuration de l'identité Git

À la toute première utilisation de Git sur une machine, il faut indiquer qui tu es : ce nom et cet e-mail seront associés à chacun de tes commits.

```
1 git config --global user.name "Prenom Nom"
2 git config --global user.email "ton.email@exemple.com"
```

Cette configuration ne se fait **qu'une seule fois par machine** : elle vaudra pour tous tes futurs projets Git, pas seulement MiniForum.

Premier commit local

```
1 git init
2 git add .
3 git commit -m "Premier commit : structure de base du MiniForum"
```

Créer un compte GitHub

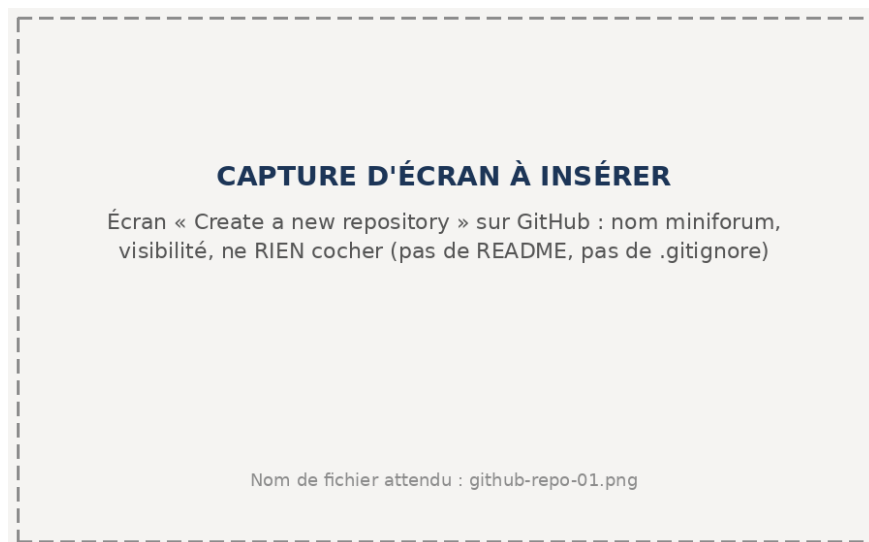
Si ce n'est pas déjà fait : aller sur `github.com`, cliquer sur *Sign up*, et suivre les étapes (e-mail, mot de passe, nom d'utilisateur).



Capture d'écran — page d'inscription GitHub

Créer le dépôt distant

1. Se connecter sur `github.com`, cliquer sur le bouton **+** en haut à droite, puis *New repository*.
2. *Repository name* : `miniforum`.
3. Laisser la visibilité en *Public* (ou *Private*, selon la consigne du cours).
4. **Ne cocher aucune case** (*Add a README file*, *.gitignore*, *licence*) : le dépôt doit rester **vide**, puisque le projet existe déjà sur ta machine.
5. Cliquer sur *Create repository*.



Capture d'écran — écran de création du dépôt GitHub

Piège fréquent

Si tu coches *Add a README file*, GitHub crée un premier commit sur le dépôt distant. Ton envoi (`git push`) sera alors refusé, car le dépôt local et le dépôt distant auront des historiques différents. Le plus simple : recommencer avec un dépôt vraiment vide.

Envoyer le projet sur GitHub

Sur la page du dépôt fraîchement créé, cliquer sur le bouton vert **Code**, onglet **HTTPS**, et copier l'URL affichée.



Capture d'écran — bouton Code, copie de l'URL HTTPS

Dans le terminal, à la racine de miniforum/ :

```
1 git remote add origin https://github.com/votre-login/miniforum.git
2 git push -u origin main
```

Lors du tout premier `git push`, une fenêtre s'ouvre automatiquement dans le navigateur et demande de se connecter à GitHub. C'est normal : GitHub n'accepte plus les mots de passe directement dans le terminal. Il suffit de se connecter une fois — Git mémorise ensuite la connexion pour les prochaines fois.



Capture d'écran — fenêtre de connexion GitHub lors du premier push

Vérification

Rafraîchir la page du dépôt sur `github.com` : les trois fichiers (`index.html`, `style.css`, `app.js`) doivent y apparaître.

Le dépôt miniform va évoluer chaque semaine. Prends l'habitude, à chaque début de séance, de récupérer les éventuelles mises à jour avec `git pull`, et de sauvegarder ton travail régulièrement avec `git add .` / `git commit` / `git push`.

5 Points de contrôle

- ☐ `node -v` et `git -version` affichent un numéro de version
- ☐ Le dossier `miniform/` contient `index.html`, `style.css`, `app.js`
- ☐ Le clic sur le bouton déclenche bien l'alerte JavaScript
- ☐ L'identité Git (`user.name` / `user.email`) est configurée
- ☐ Le dépôt Git local contient au moins un commit
- ☐ Le dépôt GitHub a été créé **vide** (sans README)
- ☐ Le dépôt miniform est visible sur GitHub avec les trois fichiers

À rendre avant la semaine 2

Lien du dépôt GitHub miniform envoyé par mail ou déposé sur la plateforme du cours.